Dynamic Programming: Ordering of matrices (in matrix-chain multiplication)

Design and Analysis of Algorithms

Dhannya SM

Ordering of matrices [1]

- Problem Statement: We are given a sequence $\langle A_1, A_2, \ldots A_n \rangle$ of n matrices to be multiplied, and we wish to compute the product $A_1.A_2.\ldots A_n$ that **minimizes the number of basic multiplications**.
- Matrix multiplication is associative.
- We can evaluate the product using the standard algorithm for multiplying pairs of matrices.
- Before that we have to parenthesize it *to decide the order in which the matrices are to be multiplied*.

[1] *Introduction to Algorithms* by Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein

Ordering of matrices

- The product $A_1.A_2.\ldots.A_n$ can be fully parenthesized in five distinct ways:
  - $A_1(A_2(A_3A_4))$
  - $A_1((A_2A_3)A_4)$
  - $(A_1A_2)(A_3A_4)$
  - $(A_1(A_2A_3))A_4$
  - $((A_1A_2)A_3)A_4$
- How we parenthesize a chain of matrices can have a dramatic impact on the cost of evaluating the product.

Example

- To illustrate the different costs incurred by different parenthesizations of a matrix product, consider the problem of a chain $\langle A_1, A_2, A_3 \rangle$ of three matrices.
- Dimensions of A_1, A_2, A_3 are 10×100, 100×5, and 5×50, respectively.
- Given below is the number of scalar multiplications for each parenthesization:
  - For $((A_1 A_2) A_3)$, we have $10 \times 100 \times 5 + 10 \times 5 \times 50 = 7500$ scalar multiplications.
  - For $(A_1 (A_2 A_3))$, we have $100 \times 5 \times 50 + 100 \times 50 \times 10 = 75000$ scalar multiplications.
- Thus, computing the product according to the first parenthesization is 10 times faster.

Optimal Substructure

- Let the notation $A_{i\ldots j}$ denote the matrix that results from evaluating the product $A_i.A_{i+1}.,\ldots,.A_j$.
- To parenthesize the product $A_i.A_{i+1}.,\ldots,.A_j$, we must split the product between A_k and A_{k+1}, for some integer k, where $i < k < j$
- That is, for some value of k, we first compute the matrices$A_{i\ldots k}$ and $A_{k+1\ldots j}$ then multiply them together to produce the final product $A_{i\ldots j}$
- The cost of parenthesizing this way is:

$$\text{the cost of computing the matrix } A_{i\ldots k}$$
$$+$$
$$\text{the cost of computing } A_{k+1\ldots j}$$
$$+$$
$$\text{the cost of multiplying them } (A_{i\ldots k} \text{ and } A_{k+1\ldots j}) \text{ together}$$

Optimal Substructure

- Suppose that to optimally parenthesize $A_i, \ldots, A_j$, we split the product between A_k and A_{k+1}.
- Then the way we parenthesize the subchain $A_i, \ldots, A_k$ within this optimal parenthesization of $A_i, \ldots, A_j$ must be an optimal parenthesization of $A_i, \ldots, A_k$.
- A similar argument applies for $A_{k+1}, \ldots, A_j$ as well

Cost of computation

- Let $m[i, j]$ be the minimum number of scalar multiplications needed to compute the matrix $A_i, \ldots, A_j$ for the full problem
- Then, the lowest cost way to compute $A_1, \ldots, A_n$ would be $m[1, n]$.
- The recurrence for this problem will be: ?

Cost of computation

- Let $m[i, j]$ be the minimum number of scalar multiplications needed to compute the matrix $A_i, \ldots, A_j$ for the full problem
- Then, the lowest cost way to compute $A_1, \ldots, A_n$ would be $m[1, n]$.
- The recurrence for this problem will be: ?

$$m[i,j] = \begin{cases} 0 & \text{if } i = j \\ \min\limits_{i \le k < j} m[i,k] + m[k+1,j] + p_{i-1}p_kp_j & \text{if } i < j \end{cases}$$

- The $m[i, j]$ values give the costs of optimal solutions to sub-problems, but they do not provide all the information we need to construct an optimal solution.

Cost of computation

- We have relatively few **distinct** sub-problems: one sub-problem for each choice of i and j giving a $\Theta(n^2)$ complexity
- We compute the optimal cost by using the tabular, bottom-up approach.
- Let the dimension of matrix A_i be $p_{i-1} \times p_i$.
- We give method $\textsc{Matrix-Chain-Order}(\mathrm{P})$, where $p = \langle p_0, p_1, \ldots, p_n \rangle$, that computes the optimal cost.

Cost of computation

```
MATRIX-CHAIN-ORDER(p)
n = p.length − 1
let m[1..n, 1..n] and s[1..n − 1, 2..n] be new tables
for i = 1 to n
    m[i, i] = 0
for l = 2 to n                    // l is the chain length
    for i = 1 to n − l + 1
        j = i + l − 1
        m[i, j] = ∞
        for k = i to j − 1
            q = m[i, k] + m[k + 1, j] + p_{i−1} p_k p_j
            if q < m[i, j]
                m[i, j] = q
                s[i, j] = k
return m and s
```

Cost of computation

- Construct the table for computing the optimal number of operations for finding the product: $A_1A_2A_3A_4A_5A_6$, where the dimensions of each A_i is given below:

$$A_1 : 3 \times 2$$
$$A_2 : 2 \times 6$$
$$A_3 : 6 \times 4$$
$$A_4 : 4 \times 5$$
$$A_5 : 5 \times 2$$

Cost of computation

$$[A_1 : 3 \times 2], [A_2 : 2 \times 6], [A_3 : 6 \times 4], [A_4 : 4 \times 5], [A_5 : 5 \times 2]$$

	1	2	3	4	5
1	0	36	72	118	116
2		0	48	88	104
3			0	120	160
4				0	40
5					0

Constructing the optimal solution

- From the s-matrix, first get the value for $s[1, n]$. Let this be k_1. Parenthesize $A_1 \ldots A_{k_1}$ and $A_{k_1+1} \ldots A_n$.
- Then, look for $s[1, k_1]$ and $s[k_1 + 1, n]$. Proceed as earlier until parenthesization has only two matrices inside for each.

```
PRINT-OPTIMAL-PARENS(s, i, j)
if i == j
    print "A"_i
else print "("
    PRINT-OPTIMAL-PARENS(s, i, s[i, j])
    PRINT-OPTIMAL-PARENS(s, s[i, j] + 1, j)
    print ")"
```

Summary

- A well-studied example of using DP for an optimization problem
- Ordering of matrices is important
- Uses the properties that matrix multiplication is associative and not commutative
- Uses the below formula to build the table in a bottom-up manner:

$$m[i,j] = \begin{cases} 0 & \text{if } i = j \\ \min\limits_{i \le k < j} m[i,k] + m[k+1,j] + p_{i-1}p_k p_j & \text{if } i < j \end{cases}$$

- Computing the ordering needs to use another recursive call to unfold the optimal path